

GIT Commands



> Version Control Like a Pro! →

★ INSTALL → `git -version` → Check Version

★ SETUP

- `git config --global user.name "Your Name"` → Set Name
- `git config --global user.email "you@email.com"` → Set Email
- `git config --list` → View Settings

★ BASIC WORKFLOW

- `git init` → Initialize Repo
- `git status` → Check Status of Files
- `git add .` → Stage All Changes
- `git status` → Check Status of Files
- `git add file.txt` → Stage One File
- `git commit -m "msg"` → Commit Changes

★ UNDO MISTAKES

- `git restore file.txt`
- `git restore --staged file.txt`
- `git reset --hard HEAD`
- `git commit --amend`
- `git branch branch-name` → Create Branch
- `git switch branch-name` → Switch Branch
- `git merge branch-name` → Merge Branches
- `git branch -d branch-name` → Delete Branch

★ BRANCHING

- `git branch branch-name`
- `git switch branch-name`
- `git merge branch-name`
- `git branch -d branch-name`

★ ADVANCED GIT

- `git stash` → Stash Changes
- `git stash list` → View Stashes
- `git stash apply` → Restore Stash

→ REMOTE REPOS

- `git clone URL` → Copy Repo
- `git fetch` → Download History
- `git pull` → Get Latest Changes
- `git push` → Upload Changes

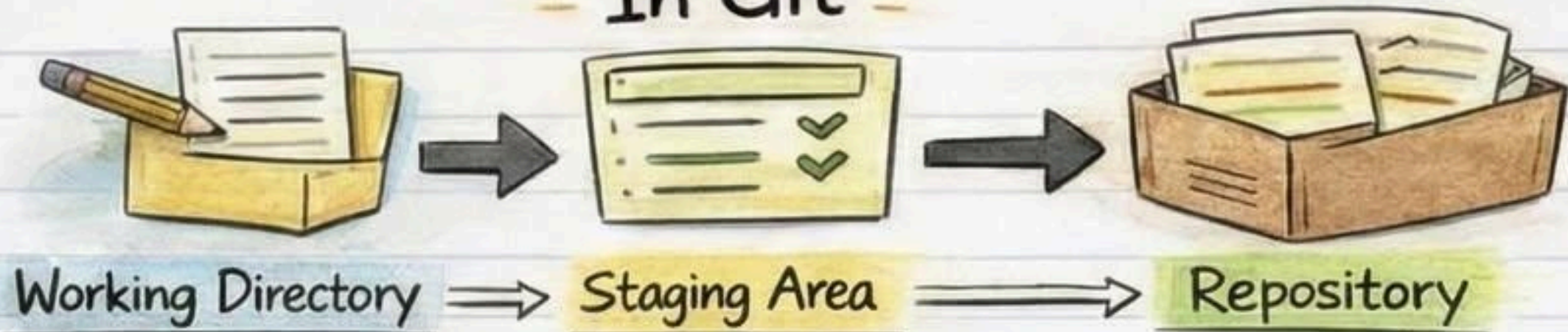
★ LOGS

- `git log` → View Full History
- `git log --oneline` → Compact History
- `git log --graph` → Branch History

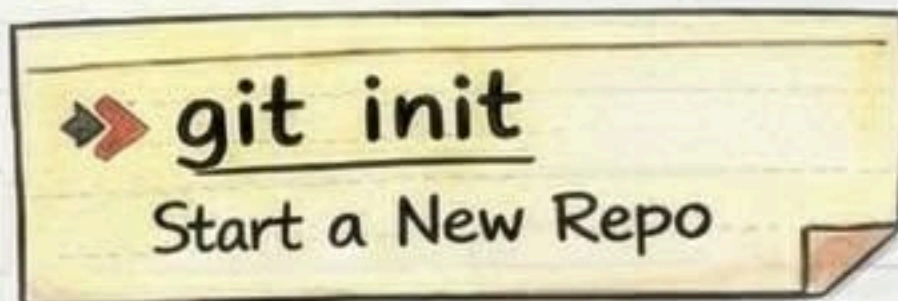
CodeWithNishchal

BASIC WORKFLOW

In Git



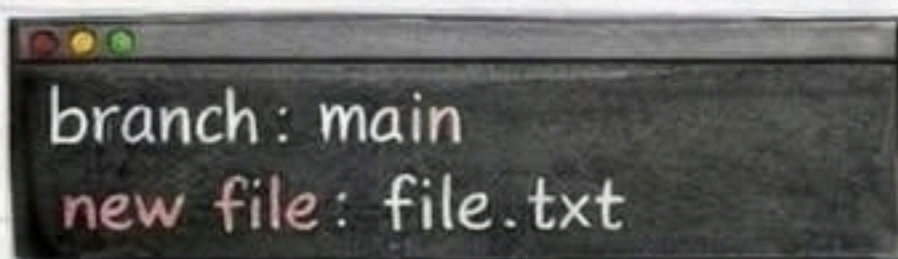
★ `git init` =



→ Start a New Repo

- Tells Git to start tracking this folder.
- Creates a hidden `.git` directory.

★ `git status` =



→ See Status of Files

- Lists which Files are modified and not yet staged.

★ `git add file.txt` =

✓ Stage One File

- "file.txt" is now staged (ready to commit)

Use this when adding one file

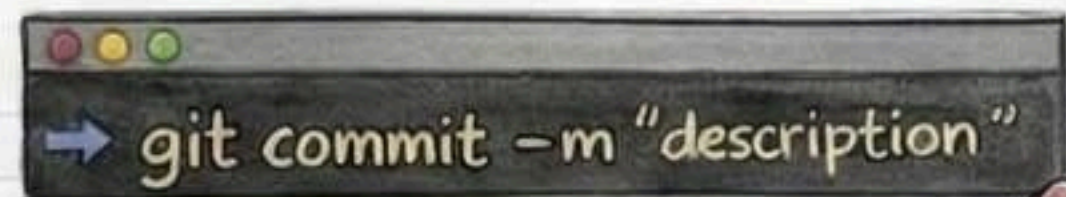


★ `git add .` =

✓ Stage All Changes

- Use when adding all changed files

Use when adding all changed files



Write descriptive message. =



★ `git commit -m "msg"` =

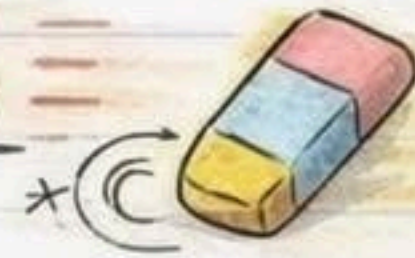
- Saves staged changes with a message explaining what you changed.

→ `git commit -m "msg"` =

- Saves staged changes with a message explaining what you changed.

UNDO MISTAKES

in Git



★ git restore file.txt

➔ Discard Changes

- Throw away your changes to "file.txt".
- Revert file to the last committed state.

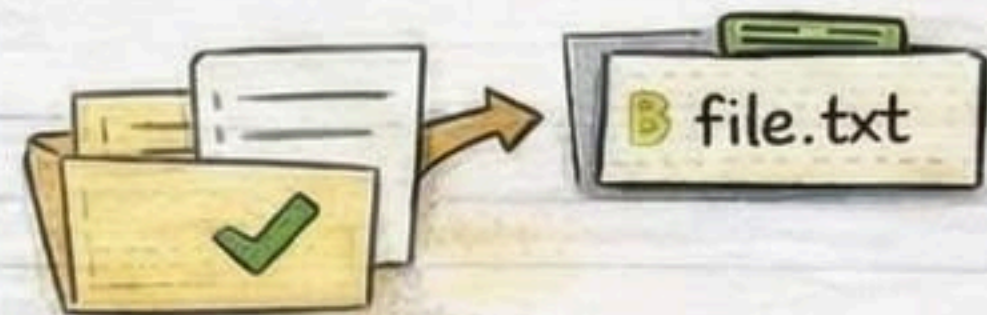


Last Commit Version

★ git restore --staged file.txt

➔ Unstage File (Keep Changes)

- Pull file.txt out of Staging Area but keep the changes.



Pull file.txt out of Staging Area but keep the changes.

★ git reset

- `git reset --soft HEAD-1`
- `git reset --mixed HEAD-1`
- ★ `git reset --hard HEAD-1`

Undo commit but keep changes staged.

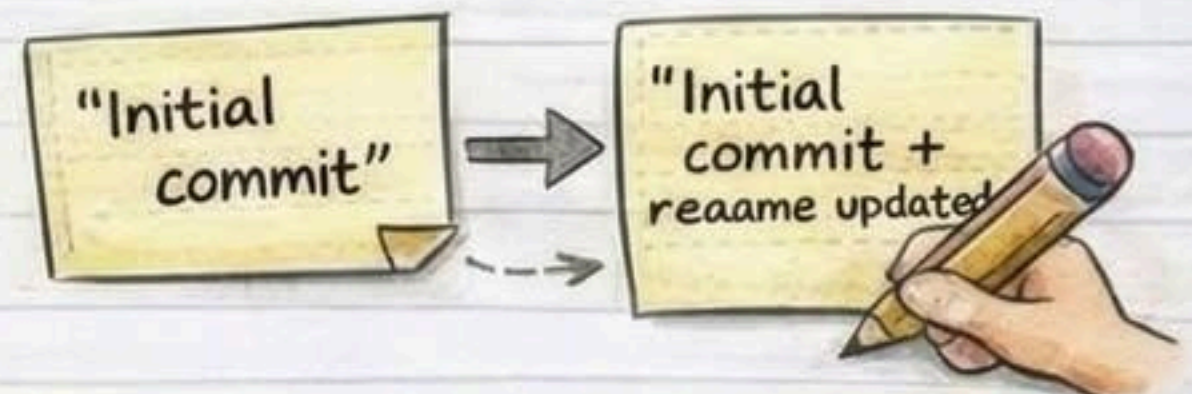
Undo commit but unstage changes.

Destroy last commit and all changes.

★ git commit --amend

➔ Edit Last Commit

- Modify the last commit with a new message or add missed files.

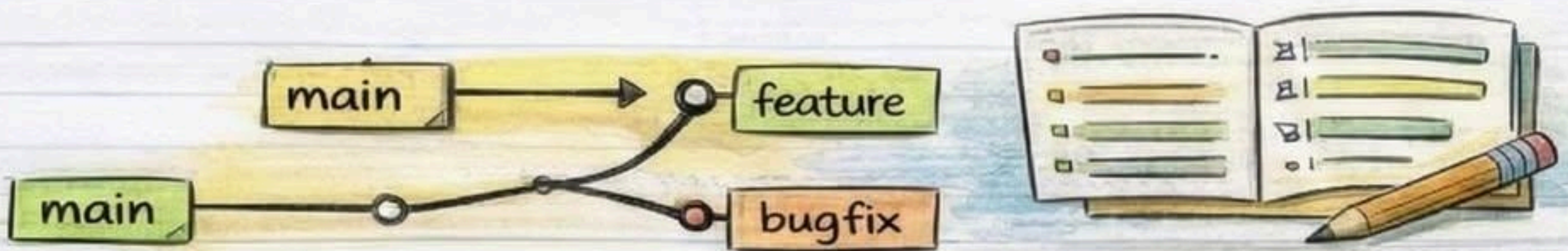


`git commit`
- `amend`

➔ `git reset`

➔ Initial commit + reame updated

BRANCHING in Git



★ **git branch branch-name** → Create Branch

→ Make a new branch for a feature.

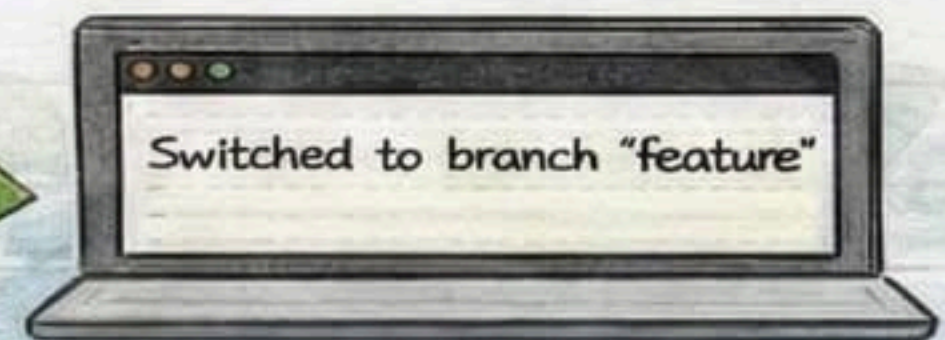
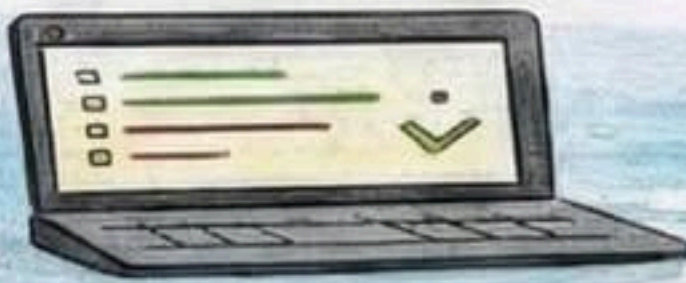
git branch
→ Create Branch



Switched to branch "feature"

★ **git switch branch-name** → Switch Branch

✓ Jump to the new branch



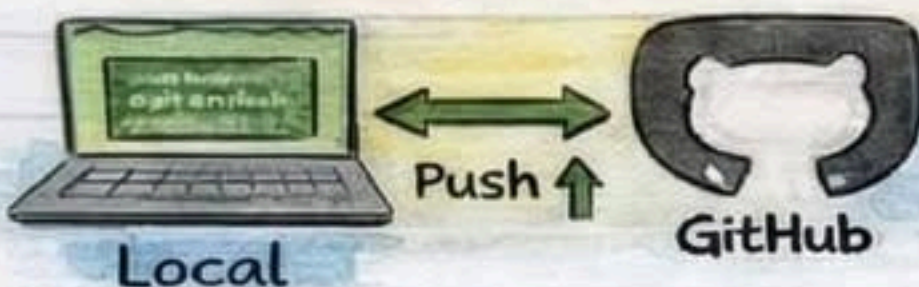
Start tracin → feature"

★ **git merge branch-name** → Merge Branches

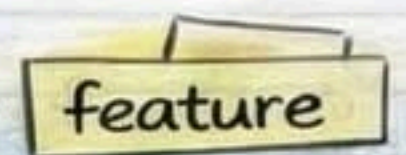
- Bring changes together with current branch.



★ **git branch -d branch-name** → Delete Branch



- Remove branch after merging.



→ **git clone URL** → Copy Repo

- Downloads a repo from GitHub

→ **git pull** → Get Latest Changes

- Fetch + merge new changes

→ **git fetch** → Download History

- Get the latest history → Review

→ **git push** → Upload Changes

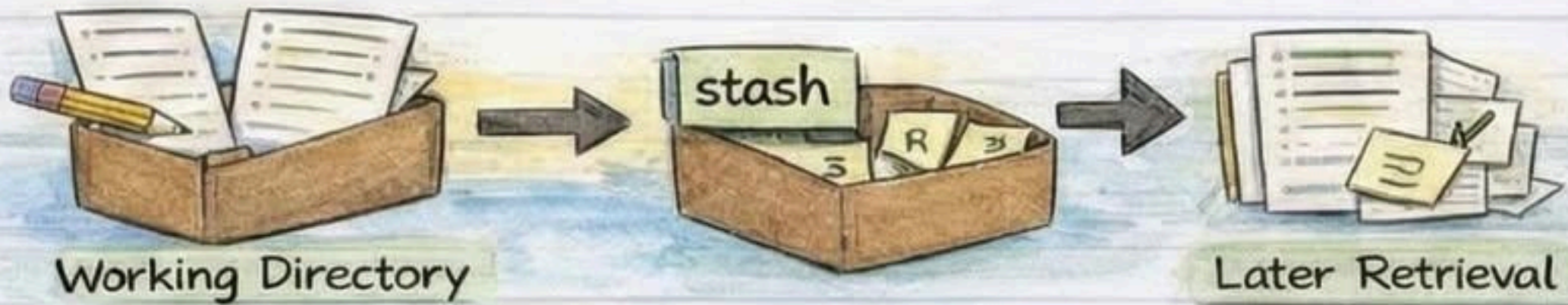
- Send commits to remote repo.

ADVANCED GIT

Commands



★ STASH CHANGES



★ git stash

→ **git stash**
→ Stash Changes

→ Stash Changes

- Temporarily save changes in a box.

→ **git stash list**
→ Stash Changes

→ View Stashes

- See all stashes stored in the box.

→ **git stash apply**
→ Restore Stash

→ Restore Stash

- Reapply the most recent stash.

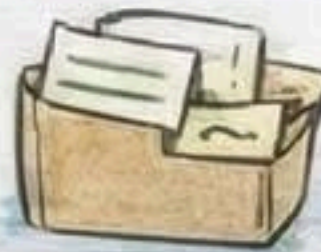
★ VIEW LOGS

→ **git log** → Full History

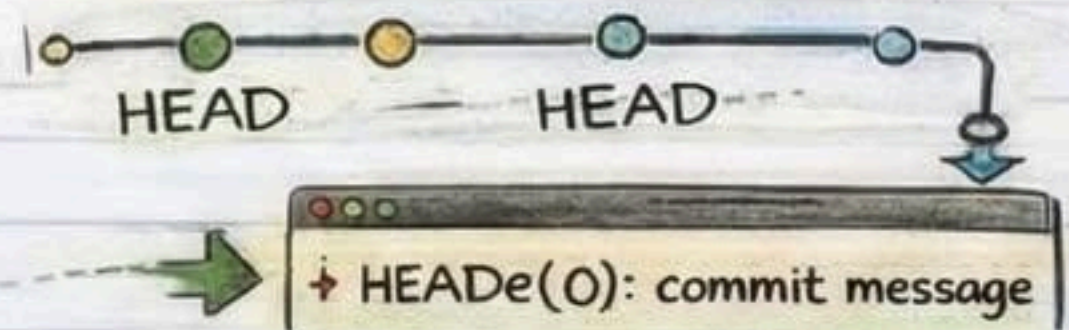
- All commits in detail.

→ **git log --oneline**

- Summarize each commit.



→ Style Logs Cleanly



→ **git log --graph**

- View branch structure visually.

→ **git log --graph**

- View branch structure visually.

★ git reflog

- Recovery Tool
- Full action history.



```
→ HEAD@0: commit message
→ HEAD@1: reset: moving to
→ HEAD@2: cherry-pick
→ HEAD@3: checkout: moving from
```